

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Mock Interview

Course Code:	ITA0517	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	Malli Chandana	Reg. No.:	192472250
Section / Batch:	CSE-AI/2024-2028	Date:	25-08-2026

Code of Conduct

I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Malli Chandana

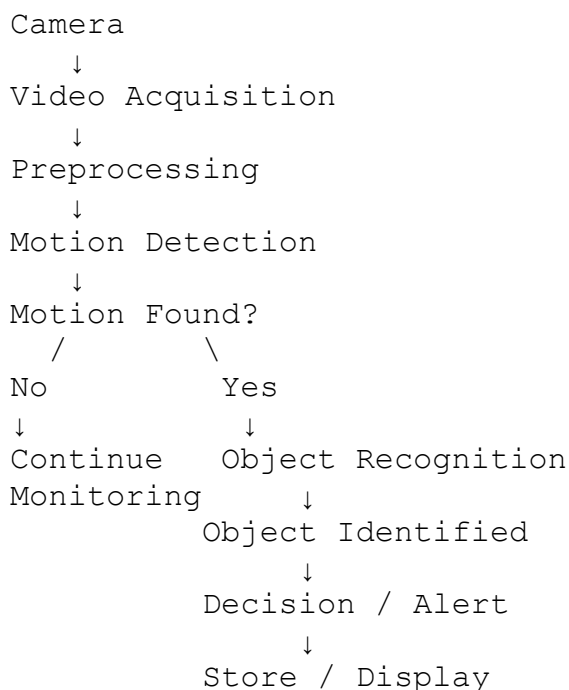
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	10	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

1. Introduction

A smart monitoring system is a computer vision system that continuously observes a camera feed, detects movement, identifies objects, and generates alerts when important events occur.

Instead of analyzing every frame with an expensive object-recognition model, the system can first perform motion detection. Object recognition is then applied only when meaningful movement is detected. This makes the system faster and more efficient.

Main idea



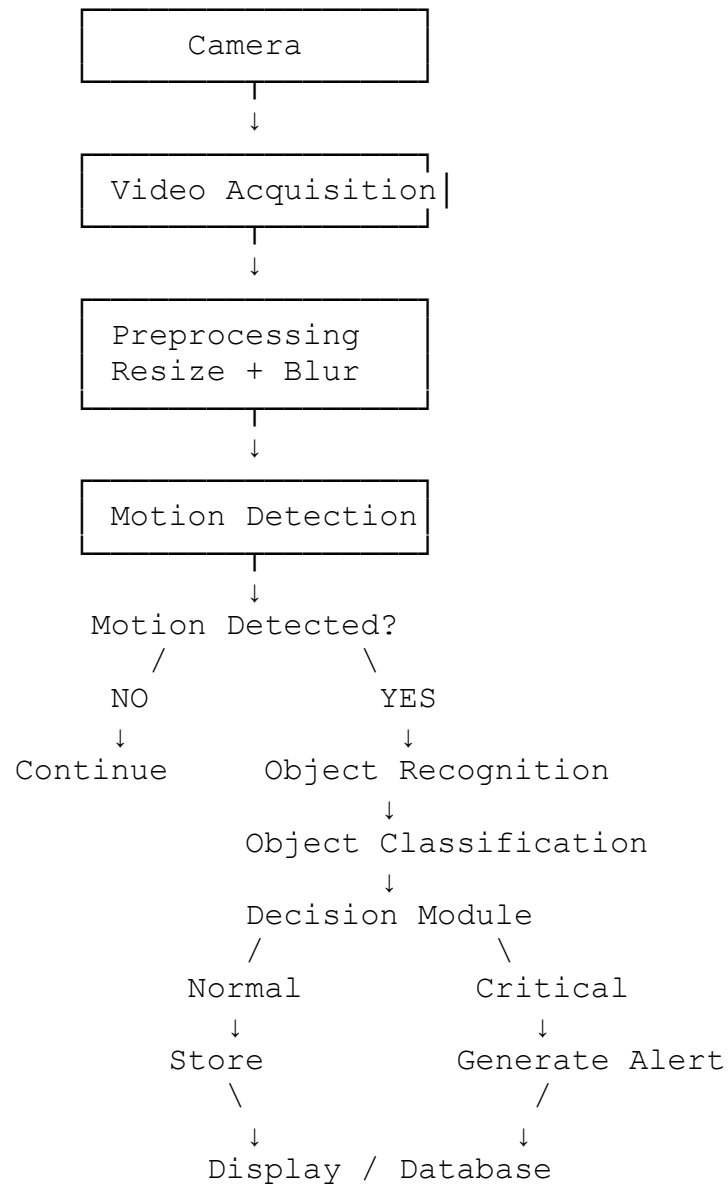
2. Objectives

The main objectives of the smart monitoring system are:

1. Capture live video continuously.
2. Detect motion in the monitored environment.
3. Identify objects present in moving regions.
4. Reduce unnecessary object-recognition computation.
5. Generate alerts for important objects or events.
6. Store relevant frames or video clips.
7. Provide real-time monitoring with low latency.

3. System Architecture

The proposed system consists of several modules.



4. Module 1 — Video Acquisition

The first module captures video from a webcam, CCTV camera, or IP camera.

Function

Camera → Video Frames

For example, a security camera may continuously generate **30 frames per second**.

Contribution to system performance

- Provides continuous monitoring.
- Determines the input frame rate.
- Camera resolution affects processing speed.
- Lower resolution can improve real-time performance.

If the camera produces unnecessarily high-resolution frames, every subsequent module has to process more pixels.

5. Module 2 — Preprocessing

The captured frames are prepared for further analysis.

Typical operations include:

- Resizing
- Grayscale conversion
- Noise reduction
- Gaussian filtering
- Brightness normalization

Example

```
Original Frame
    ↓
Resize
    ↓
Grayscale
    ↓
Gaussian Blur
    ↓
Processed Frame
```

Contribution to performance

Preprocessing reduces noise and computational requirements.

For example, converting a color image into grayscale reduces three color channels to one intensity channel for motion analysis.

This can improve:

- Processing speed
- Motion detection accuracy
- Noise tolerance

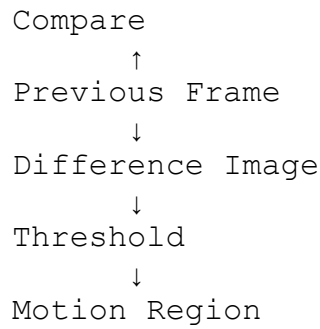
6. Module 3 — Motion Detection

Motion detection identifies whether something has changed between consecutive frames.

A simple approach is **frame differencing**.

Process

```
Current Frame
    ↓
```



The difference can be calculated as:

$$D(x, y) = | I_t(x, y) - I_{t-1}(x, y) |$$

where:

- I_t = current frame
- I_{t-1} = previous frame
- $D(x, y)$ = difference between frames

If the difference is greater than a threshold, the pixel can be considered part of a moving region.

Contribution to performance

This is one of the most important optimization modules.

Instead of running object recognition on **every frame**, the system first asks:

"Is there any movement?"

If there is no motion, the expensive recognition model can be skipped.

This significantly reduces:

- CPU/GPU usage
- Processing time
- Power consumption
- Unnecessary model inference

7. Module 4 — Noise Removal

Motion detection can produce false detections because of:

- Camera noise
- Small lighting changes
- Shadows
- Moving leaves
- Rain

- Reflections

Morphological operations can help clean the motion mask.

Example

Motion Mask

↓

Erosion

↓

Dilation

↓

Clean Motion Region

Common operations include:

- Opening
- Closing
- Erosion
- Dilation

Contribution to performance

Noise removal reduces false motion events.

This prevents the object-recognition module from being unnecessarily triggered.

8. Module 5 — Motion Region Extraction

Once motion is detected, the system identifies the region where movement occurred.

Contours or connected components can be used.

Frame

↓

Motion Mask

↓

Contours

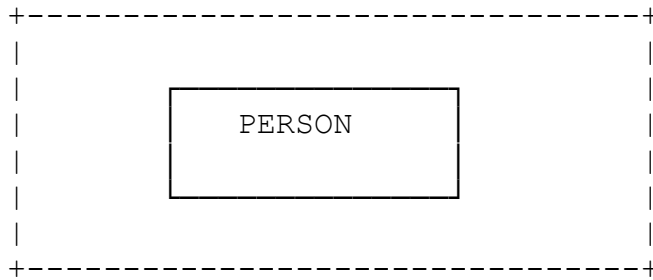
↓

Bounding Boxes

↓

Moving Region

For example:



Contribution to performance

Instead of passing the **entire frame** to the object-recognition model, only the moving region can be processed.

This is called **Region of Interest (ROI) processing**.

It reduces the number of pixels analyzed by the recognition model and can significantly improve speed.

9. Module 6 — Object Recognition

When motion is detected, the system identifies the object responsible for the movement.

Possible models include:

- YOLO
- SSD
- Faster R-CNN
- MobileNet-SSD

For a real-time system, lightweight object detectors such as **YOLO variants or MobileNet-based detectors** are often appropriate depending on the hardware and accuracy requirements.

Example

The camera detects movement:

Motion Detected



Object Recognition



Person	92%
Dog	87%
Car	94%

Contribution to performance

Object recognition provides **semantic information**.

Motion detection tells us:

"Something moved."

Object recognition tells us:

"A person moved."

or

"A vehicle moved."

This distinction makes the monitoring system intelligent rather than simply motion-sensitive.

10. Module 7 — Object Tracking

Once an object has been detected, a tracking algorithm can follow it across subsequent frames.

Examples:

- Centroid tracking
- SORT
- Deep SORT
- Optical flow

Process

```
Detection
  ↓
Object ID = 1
  ↓
Track Object
  ↓
Frame 1 → Frame 2 → Frame 3 → Frame 4
```

Contribution to performance

Tracking reduces the need to run full object detection on every frame.

For example:

```
Frame 1 → Detection
Frame 2 → Tracking
Frame 3 → Tracking
Frame 4 → Tracking
Frame 5 → Detection
```

This can significantly improve real-time performance.

11. Module 8 — Decision and Alert Module

The decision module determines whether the detected event is important.

For example:

```
Object = Person
      +
Motion = Detected
      +
Restricted Area = YES
      ↓
      ALERT
```

Possible alerts include:

- Popup notification
- Email
- SMS
- Alarm
- Mobile notification
- Recorded video

Example

If the system is monitoring a restricted laboratory:

```
Person detected
      ↓
Restricted area?
      ↓
      YES
      ↓
Generate Alert
      ↓
Save Image/Video
```

Contribution to performance

The decision module prevents unnecessary alerts by applying rules before notification.

This reduces **false alarms** and makes the monitoring system more useful.

12. Module 9 — Recording and Storage

The system can store:

- Timestamp
- Detected object
- Confidence score

- Bounding box
- Image
- Short video clip

Example:

Time: 10:32:15
Object: Person
Confidence: 94%
Event: Motion detected
Location: Entrance

Contribution to performance

Instead of continuously storing the entire camera feed, the system can store only important events.

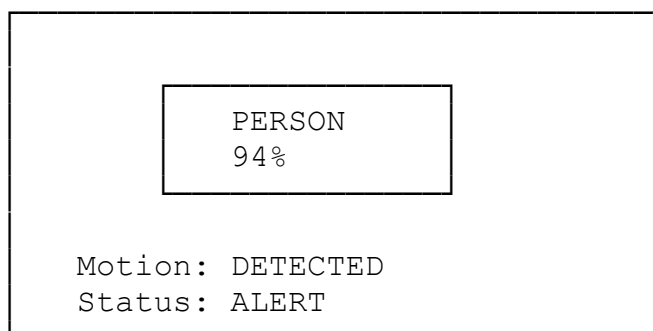
This reduces:

- Storage requirements
- Network bandwidth
- Database size

13. Module 10 — Visualization

The final result can be displayed with bounding boxes and labels.

Example:

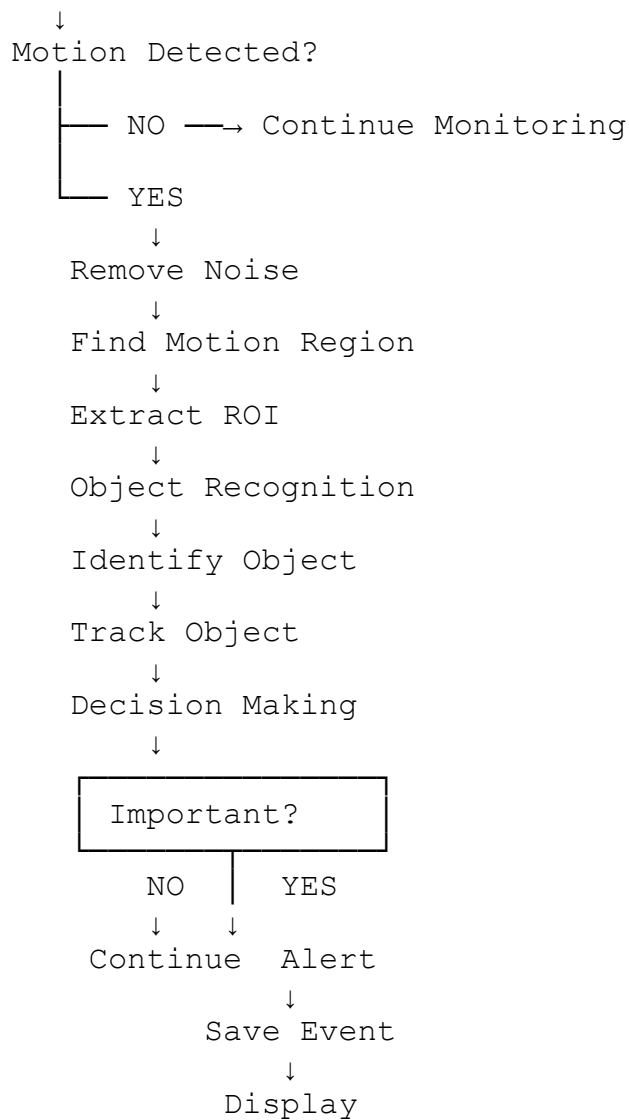


The display helps the operator understand what is happening in real time.

14. Complete Processing Pipeline

```
START
↓
Capture Video
↓
Preprocess Frame
↓
```

Compare Current & Previous Frame



15. How Each Module Improves Performance

Module	Main Function	Contribution
Video Capture	Acquire frames	Provides continuous input
Preprocessing	Clean and resize frames	Reduces noise and computation
Motion Detection	Detect changes	Avoids unnecessary recognition
Noise Removal	Clean motion mask	Reduces false detections
ROI Extraction	Select moving region	Reduces pixels processed
Object Recognition	Identify objects	Provides semantic information
Object Tracking	Follow detected objects	Reduces repeated detection
Decision Module	Evaluate events	Reduces false alerts
Storage	Save important events	Reduces storage requirements
Visualization	Show results	Enables real-time monitoring

16. Optimization Strategies

1. Use Motion Detection First

Object recognition is computationally expensive. Therefore:

```
Motion Detection
      ↓
If motion exists
      ↓
Object Recognition
```

This is more efficient than:

```
Every Frame
      ↓
Object Recognition
```

2. Use ROI Processing

Only analyze areas where movement is detected.

```
Full Frame
      ↓
Motion Detection
      ↓
ROI
      ↓
Object Recognition
```

This reduces computational workload.

3. Use a Lightweight Model

For real-time monitoring, a lightweight detector can provide a better speed/accuracy balance than a very large model.

4. Reduce Frame Resolution

For example:

```
1920 × 1080
      ↓
640 × 480
```

Lower resolution reduces processing time, although excessive reduction can decrease recognition accuracy.

5. Frame Skipping

The system can process selected frames instead of every frame.

Frame 1 → Process
Frame 2 → Skip
Frame 3 → Process
Frame 4 → Skip

This reduces computational load.

6. Multithreading

Different tasks can run independently:

Thread 1 → Capture
Thread 2 → Motion Detection
Thread 3 → Object Recognition
Thread 4 → Display

This can improve responsiveness when hardware supports parallel processing.

17. Example Scenario

Consider a **college campus entrance**.

The camera continuously monitors the entrance.

Situation 1 — No movement

Camera
↓
Motion Detection
↓
No Motion
↓
Skip Object Recognition

Very little computation is required.

Situation 2 — Person enters

Camera
↓
Motion Detected
↓
Motion Region
↓
Object Recognition
↓
Person Detected
↓

Check Rules
↓
Record Event

Situation 3 — Vehicle enters restricted area

Motion Detected
↓
Object Recognition
↓
Vehicle Detected
↓
Restricted Area = YES
↓
ALERT
↓
Save Image + Timestamp

This demonstrates why combining motion detection and object recognition is more intelligent than using either module alone.

18. Advantages

- Real-time monitoring
- Reduced computational cost
- Lower false alerts
- Automatic object identification
- Efficient storage
- Can operate continuously
- Supports multiple object classes
- Can be integrated with tracking and alert systems
- Suitable for edge devices and surveillance applications

19. Limitations

- Poor lighting can affect motion detection.
- Camera movement can cause false motion.
- Shadows may be incorrectly detected as movement.
- Object recognition models require computational resources.
- Occlusion can make object recognition difficult.
- Very low-resolution cameras can reduce recognition accuracy.
- Thresholds and detection parameters may require tuning.

20. Applications

The system can be used in:

1. Home security
2. College and school surveillance
3. Industrial monitoring

4. Traffic monitoring
5. Parking management
6. Warehouse security
7. Retail stores
8. Restricted-area monitoring
9. Smart cities
- 10. Wildlife monitoring**

21. Conclusion

A smart monitoring system that combines motion detection and object recognition provides an efficient approach to intelligent video surveillance. Motion detection acts as a first-level filter, identifying whether meaningful activity has occurred. Object recognition then provides detailed information about what caused the activity.

The combination improves system performance because expensive recognition operations are performed only when required. ROI processing, lightweight models, frame skipping, tracking, and efficient preprocessing further reduce computational requirements.

Therefore, the proposed architecture achieves a good balance between accuracy, speed, resource utilization, and real-time responsiveness, making it suitable for modern surveillance and monitoring applications.